Bloco · Algoritmos · Tabelas hash (8 questões · diagnóstico)

> Questões autorais no estilo FGV, elaboradas para treino. Não são questões oficiais.

> Prova diagnóstica, feita antes da teoria, para medir o que ficou da faculdade. Assunto único: tabelas hash. Marque "?" no que não souber e pontinho nas candidatas. Nunca deixe em branco.

Questão 1

O sistema e-GUIAS usa uma tabela hash com 7 posições (0 a 6), função de hash h(k) = k mod 7 e tratamento de colisões por encadeamento separado, em que cada nova chave é inserida **no fim** da lista da sua posição. O analista Otávio inseriu, nesta ordem, as chaves 17, 8, 10, 14, 24 e 20.

A posição com mais chaves e a sequência da lista nessa posição são:

(A) posição 1, com 8 → 17 → 24;

(B) posição 3, com 24 → 10 → 17;

(C) posição 4, com 10 → 17 → 24;

(D) posição 3, com 17 → 10 → 24;

(E) posição 3, com 10 → 17 → 24.

Questão 2

A analista Isabela implementou, no sistema e-PAUTA, uma tabela hash com 10 posições (0 a 9), função de hash h(k) = k mod 10 e endereçamento aberto com sondagem linear (em caso de colisão, tenta-se a posição seguinte, voltando ao 0 após o 9). A tabela estava vazia, e foram inseridas, nesta ordem, as chaves 31, 41, 52, 11 e 60.

A posição ocupada pela chave 11 e o número de posições examinadas para inseri-la, contando a posição inicial, são, respectivamente:

(A) posição 1 e 1 posição examinada;

(B) posição 4 e 4 posições examinadas;

(C) posição 2 e 2 posições examinadas;

(D) posição 4 e 3 posições examinadas;

(E) posição 5 e 5 posições examinadas.

Questão 3

O cache de consultas do sistema e-CONSULTA é uma tabela hash com 2.000 posições e tratamento de colisões por encadeamento separado. Em determinado momento, a tabela armazena 1.500 chaves, distribuídas de modo uniforme pela função de hash. O analista Hugo precisa informar o fator de carga da tabela à equipe.

O fator de carga e a sua interpretação correta são:

(A) 1,33, e, acima de 1, a tabela com encadeamento deixa de aceitar novas chaves;

(B) 0,75, e, com encadeamento, o fator de carga nunca pode passar de 1;

(C) 0,75, e o tempo médio de busca passa a ser O(n), o que exige trocar a tabela por uma árvore;

(D) 0,25, que é a fração de posições ainda livres na tabela;

(E) 0,75, que é o número médio de chaves por posição, e o redimensionamento (rehash) mantém a busca média em O(1).

Questão 4

A equipe da analista Mirela desenvolveu uma tabela hash própria para o sistema e-CUSTAS, com tratamento de colisões por encadeamento em listas encadeadas simples, não ordenadas. Por erro de implementação, a função de hash da classe Processo devolve sempre o valor 42, qualquer que seja o processo. A tabela armazena n processos.

A complexidade de tempo da busca de um processo nessa tabela, no pior caso, é:

(A) O(n), pois todas as chaves caem na mesma posição, e a busca percorre uma única lista com n elementos;

(B) O(1), pois a tabela hash garante acesso constante, independentemente da função de hash;

(C) O(log n), pois a tabela mantém ordenada a lista de chaves que colidem;

(D) O(1), pois o método equals() compara os objetos diretamente, sem percorrer a lista;

(E) $O(n^2)$, pois cada busca recalcula o hash de todos os elementos armazenados.

Questão 5

O analista Leandro escreveu a classe a seguir, em Java, para o sistema e-PROCESSO, e testou o seu uso em um HashSet:

```
class Processo {
    String numero;

    Processo(String numero) { this.numero = numero; }

    @Override
    public boolean equals(Object o) {
        return o instanceof Processo p && numero.equals(p.numero);
    }
    // hashCode() não foi sobrescrito
}

Set<Processo> s = new HashSet<>();
s.add(new Processo("0801234-56.2026.8.15.2001"));
s.add(new Processo("0801234-56.2026.8.15.2001"));
System.out.println(s.size());
```

O valor impresso e a sua causa são:

(A) 1, pois equals() foi sobrescrito, e o HashSet usa apenas equals() para detectar elementos repetidos;

(B) 1, pois o HashSet compara os números de processo diretamente como String;

(C) 2, pois objetos iguais pelo equals() precisam ter o mesmo hashCode(), e, sem sobrescrevê-lo, os dois objetos tendem a cair em posições diferentes e não são comparados;

(D) 2, pois o HashSet sempre permite elementos repetidos quando eles são objetos, e não tipos primitivos;

(E) 0, pois o código lança exceção ao inserir um objeto cuja classe não sobrescreve hashCode().

Questão 6

A analista Natália precisa atender a duas demandas do sistema e-DISTRIBUI, escrito em Java:

1. localizar um processo pelo número exato, com milhões de consultas por dia;
2. listar, em ordem, os processos distribuídos entre duas datas informadas pelo usuário.

As estruturas mais adequadas para as demandas 1 e 2 são, respectivamente:

(A) árvore de busca balanceada e tabela hash;

(B) tabela hash e tabela hash, pois as duas consultas são O(1);

(C) lista encadeada e tabela hash;

(D) pilha e árvore de busca balanceada;

(E) tabela hash e árvore de busca balanceada, como a TreeMap.

Questão 7

O analista Wagner escreveu o trecho a seguir, em Python, para guardar em cache a situação de um processo, usando como chave o número e o ano:

```
cache = {}
chave = ["0801234", 2026]
cache[chave] = "concluso"
```

Ao executar o trecho, o interpretador:

(A) armazena o valor normalmente, pois qualquer objeto Python pode ser usado como chave de dicionário;

(B) lança TypeError, pois a lista é mutável e não pode ser chave de dicionário, e uma tupla resolveria o problema;

(C) lança KeyError, pois a chave ainda não existia no dicionário no momento da atribuição;

(D) armazena o valor, mas converte automaticamente a lista em string para usá-la como chave;

(E) lança TypeError, pois as chaves de dicionário precisam ser strings, e uma tupla também falharia.

Questão 8

O sistema e-INTIMA usa uma tabela hash com endereçamento aberto e sondagem linear. Para remover uma chave, o analista Diego simplesmente marcou a posição como vazia. A partir daí, buscas por algumas chaves que continuavam na tabela passaram a informar que elas não existiam.

A explicação correta para o problema é que:

(A) a sondagem linear não permite remoção, e a tabela precisa ser recriada a cada exclusão;

(B) a função de hash mudou após a remoção, pois depende da quantidade de chaves armazenadas;

(C) com a posição esvaziada, o fator de carga ficou acima de 1, e a tabela deixou de funcionar;

(D) a busca para ao encontrar uma posição vazia, e, ao esvaziar a posição, chaves deslocadas para depois dela ficam inalcançáveis, por isso se usa uma marca de "removido";

(E) a tabela passou a usar encadeamento separado após a remoção, e as listas das posições ficaram vazias.

Minhas respostas

Questão	Resposta
1	
2	
3	
4	
5	
6	
7	
8	